Context Compression Engine

AI/ML Build Guide — Revised (Zero-Cost Architecture) · v2.0

What changed from v1.0 and why

All four prompts were originally designed and tested against Claude. In v2.0, the two high-volume prompts (chunk scoring, chunk compression) move to Groq's Llama 3.3 70B to stay at zero cost. The two low-volume prompts (question generation, similarity judging) stay on Claude Haiku 4.5, with Gemini 2.5 Flash as a documented fallback. Re-test all four prompts against their new model before handing off to Backend — Llama and Claude don't always follow formatting instructions the same way.

Prompt	v1.0 model	v2.0 model	Notes
Chunk scoring	Claude	Groq — Llama 3.3 70B	High volume; re-test JSON reliability
Chunk compression	Claude	Groq — Llama 3.3 70B	High volume; conservative code variant unchange
Question generation	Claude	Claude Haiku 4.5	Low volume, one call per run
Similarity judging	Claude	Claude Haiku 4.5	Low volume, judge-facing — quality matters most

1. Your responsibilities (unchanged)

- Design and test the chunk scoring prompt on Groq/Llama 3.3 70B
- Design and test the chunk compression/summarization prompt on Groq/Llama 3.3 70B
- Design and test the question generation prompt on Claude Haiku 4.5
- Design and test the similarity judging prompt on Claude Haiku 4.5
- Tune thresholds (score cutoffs, chunk size, batch size) based on real test runs
- Document fallback behavior and hand off finalized prompts to Backend

2. Test samples (unchanged from v1.0)

- A code file (~150-300 lines) with repeated imports/boilerplate
- A log file (~200+ lines) with repeated timestamp/header patterns
- A long chat/support transcript (~100+ messages) with filler ("thanks", "ok", "got it")

3. Chunk scoring prompt — now on Groq/Llama 3.3 70B (1 hr)

Same goal as v1.0: return a 1-10 relevance score per chunk. Prompt structure is the same; what to check changes slightly for Llama:

```
You are scoring chunks of text for information density and importance for future retrie
val.
For each chunk below, assign a score from 1 (pure filler/boilerplate, safe to remove) t
o
10 (critical unique information -- specific facts, numbers, logic, named entities -- mu
st
be kept).

Return ONLY valid JSON, no other text, in this exact format:
[{"id": "chunk_1", "score": 7, "reason": "contains function logic and variable names"},
 ...]

Do not include any text before or after the JSON array. Do not wrap the JSON in markdow
n
code fences.

Chunks:
{chunk_1_id}: {chunk_1_text}
{chunk_2_id}: {chunk_2_text}
...
```

- Llama models more often wrap JSON in ```json fences despite instructions -- strip fences before parsing rather than treating it as a failure
- Set temperature to 0 for scoring calls -- consistency matters more than creativity here
- Start at ~10 chunks per batch call, same as v1.0; Groq's free-tier TPM cap (~6,000) may force smaller batches for large chunks -- measure and adjust
- If JSON parsing still fails after one retry, apply the documented fallback: treat all chunks in that batch as score 5

4. Chunk compression prompt — now on Groq/Llama 3.3 70B (1 hr)

```
Compress the following text as much as possible while strictly preserving:
- All named entities (people, systems, variables, function names)
- All numbers, dates, and quantities
- All logical/causal relationships (if X then Y)
Remove filler words, redundant phrasing, and repeated explanations. Do not add new
information. Return only the compressed text, no preamble.

Text:
{chunk_text}
```

- For code chunks specifically: only compress comments/docstrings, never the code logic itself -- route code chunks through this more conservative variant, same rule as v1.0
- Verify numbers/entities are preserved exactly, not paraphrased or rounded -- check this deliberately against all 3 sample types, Llama models can be more liberal with rounding than Claude was in testing
- Confirm combined Stage 1 + Stage 2 compression still hits the 70% target -- if Llama's summarization is less aggressive than Claude's was, lower the score threshold (e.g. <7 instead of <6) to compress more chunks

5. Question generation prompt — Claude Haiku 4.5 (30 min, mostly unchanged)

```
Given the following content, generate ONE specific question that:
- Can only be answered correctly by someone who read this content
- Tests a specific fact, number, or piece of logic (not a vague/general question)
- Has a clear, checkable answer
Return only the question, no preamble.

Content:
{original_text}
```

Low volume, stays on Claude for reliability -- one call per run, cost is negligible.

6. Similarity judging prompt — Claude Haiku 4.5, judge-facing (45 min)

```
Compare these two answers to the same question and rate how semantically similar/
consistent they are on a scale of 0-100, where 100 means they convey identical factual
content (wording can differ) and 0 means they are contradictory or unrelated.

Question: {question}
Answer A: {answer_full}
Answer B: {answer_compressed}

Return ONLY valid JSON in this format: {"score": 87, "reasoning": "brief explanation"}
```

- This is the number and reasoning judges will see directly -- keep it on Claude for quality, don't move it to Groq even though it's tempting for cost reasons

- Calibrate with at least 5 test pairs: two near-identical answers should score 90+, two answers with a clear factual contradiction should score under 40
- If Claude's $5 credit is exhausted before demo day, swap in Gemini 2.5 Flash with the same prompt -- test this fallback path once in advance, don't discover it live

7. Threshold tuning (1 hr, do after Backend integrates Steps 3-6)

- Score threshold for compression -- may need to shift from v1.0's assumed <6, since Llama's scoring distribution can skew differently than Claude's; re-tune against real compression ratio results
- Chunk size -- same tuning process as v1.0 (smaller chunks for more granular scoring, larger for more context per decision)
- Batch size for Groq scoring calls -- now also bounded by the free-tier TPM cap, not just quality/attention degradation, so tune with rate limits in mind
- Document final values and why -- still good pitch-deck material ("we tuned X and found Y")

8. Fallback behavior (30 min) — updated for rate limits

- JSON parsing fails on the scoring prompt -> treat all chunks in that batch as score 5, apply light compression uniformly
- A chunk compression call times out -> keep that chunk verbatim (under-compress rather than fail the run)
- Groq returns a 429 (rate limited) -> queue and retry once after a short backoff, then fall back to Stage-1-only if still failing
- Claude validation call fails for any reason (rate limit, exhausted credit, timeout) -> retry once, then fall back to Gemini with the identical prompt

What "done" looks like

- Four finalized, tested prompts: two running on Groq/Llama 3.3 70B, two on Claude Haiku 4.5
- Documented expected input/output JSON schema for each
- Documented threshold values (score cutoff, chunk size, batch size) re-tuned for the new models
- Documented fallback behavior including rate-limit handling
- All four prompts manually tested against all 3 sample content types on their new model

Coordination notes

- Hand off each prompt to Backend as soon as it's re-tested on its new model -- don't wait for all four
- Flag early if Llama's JSON reliability or compression aggressiveness seems inconsistent -- better to redesign at hour 6 than discover it at hour 20
- Keep a shared doc of example inputs -> outputs per prompt per model, so Backend can write good test fixtures for both Groq and Claude paths